Documentación del Proyecto: SecurePlan Mobile (Calendario) Generado: 2026-10-15 ## 1. Resumen Aplicación móvil de agenda con frontend React Native + Expo y backend Node.js + Express. PostgreSQL como base de datos, JWT para autenticación y Expo SecureStore para almacenamiento seguro de tokens. Se aplican principios de arquitectura limpia, codificación segura e IDOR mitigation. ## 2. Objetivo de esta documentación Documentar lo implementado según lo solicitado en Sprint 1 y Sprint 2, identificando dónde se aplican los patrones, qué comandos se utilizaron y evidenciando la ubicación de cada parte en el código. ### Sprint 1 — Base Móvil, Arquitectura y Autenticación Segura - **Backend:** conexión a BD con patrón **Singleton**, módulo de usuarios con cifrado asíncrono (bcrypt/Argon2) y generación de tokens **JWT** firmados. - **Frontend:** arquitectura limpia de directorios, navegación protegida (pilas separadas para autenticados/no autenticados) y formularios de Login/Registro. - **Seguridad:** prohibido el texto plano en BD y el JWT en almacenamiento común; se usa **Expo SecureStore**. ### Sprint 2 — Core de la Agenda (Módulo de Eventos) - **Backend:** endpoints CRUD de eventos protegidos por middleware (título, descripción, fecha, hora, categoría, prioridad) con validación estricta mediante esquemas. - **Frontend:** vista principal con componente de calendario e interceptores HTTP globales que adjuntan el JWT automáticamente. - **Seguridad:** mitigación de vulnerabilidades **IDOR** (un usuario no puede acceder a eventos de otro). ## 3. ¿Dónde se aplicó el Patrón Singleton? ### Backend - Conexión a Base de Datos **Archivo:** `backend/src/config/database.js` Explicación: Se crea una única instancia de Pool de PostgreSQL (singleton). Si ya existe `pool` global, se reutiliza; si no, se inicializa con las variables de entorno. Esto garantiza una única conexión compartida a la BD. Código relevante: ```js const { Pool } = require('pg'); const env = require('./env'); let pool; if (!pool) { pool = new Pool({ host: env.db.host, ... }); } module.exports = { query: (text, params) => pool.query(text, params), pool }; ``` ## 4. Arquitectura Limpia - Frontend **Estructura implementada en `frontend/src/*:** - `domain/*: entidades (`Event.js`, `User.js`), constantes (`eventMeta.js`), utilidades (`date.js`). Capa de dominio pura. - `data/*: cliente HTTP (`api/client.js`), repositorios (`repositories/authRepository.js`, `eventsRepository.js`). Abstracción de acceso a datos. - `services/*: almacenamiento seguro (`secureStorage.js`) - abstrae Expo SecureStore. - `navigation/*: `RootNavigator.js` (lógica de auth vs app), `AuthStack.js`, `AppStack.js` - capa de navegación. - `presentation/*: componentes reutilizables, pantallas (Login/Register/Agenda/Splash), contexto (`AuthContext.js`, `EventsContext.js`) - UI. - `theme/*: `theme.js` - estilos globales. ## 5. Navegación protegida (pilas separadas) **RootNavigator** (`frontend/src/navigation/RootNavigator.js`): determina si el usuario está autenticado o restaurando. Muestra `SplashScreen` mientras `isRestoring` es true. Si autenticado → `AppStack` envuelto en `EventsProvider`. Si no → `AuthStack`. **AuthStack** (`frontend/src/navigation/AuthStack.js`): pila no autenticada con pantallas `Login` y `Register`. ```js createNativeStackNavigator({ initialRouteName: 'Login', screenOptions: { headerShown: false } }) ``` **AppStack** (`frontend/src/navigation/AppStack.js`): pila autenticada con `Agenda`. **AuthContext** (`frontend/src/presentation/context/AuthContext.js`): gestiona `isAuthenticated`, `isRestoring`, `user`, `token`, funciones `signIn`, `signUp`, `signOut`. Restaura token de SecureStore al iniciar. ## 6. Sprint 1 - Lo implementado (detalle) ### Backend - `backend/src/config/env.js`: carga variables de entorno (.env) con validación mínima. - `backend/src/config/database.js`: patrón Singleton para Pool de PostgreSQL. - `backend/src/models/user.model.js`: modelo User, normaliza email (lowercase), hash asíncrono con `bcrypt` (SALT_ROUNDS=12), método `verifyPassword` con `bcrypt.compare`. - `backend/src/services/token.service.js`: firma/verifica JWT (payload {sub, email}, `expiresIn` desde env). - `backend/src/controllers/auth.controller.js`: `register` valida email/contraseña (mín 8), evita duplicados (409), crea usuario, devuelve {user, token}. `login` verifica credenciales (401 si falla), devuelve token. `me` requiere auth. - `backend/src/middleware/auth.middleware.js`: extrae Bearer token y verifica con `tokenService.verify()`. ### Frontend - `frontend/src/services/secureStorage.js`: guarda/obtiene/elimina JWT en Expo SecureStore (clave `AUTH_TOKEN`). - `frontend/src/data/api/client.js`: instancia axios con `baseURL` desde `EXPO_PUBLIC_API_URL` (usa 10.0.3.2 para Genymotion), interceptor de request añade `Authorization: Bearer `, interceptor de response borra token ante 401. - `frontend/src/data/repositories/*`: repositorios que usan `apiClient` para auth y eventos. - `frontend/src/presentation/screens/LoginScreen.js`, `RegisterScreen.js`: formularios con validación cliente (regex email, min password 8, confirm match). - `frontend/src/presentation/context/AuthContext.js`: flujo completo de auth + restauración desde SecureStore. - `frontend/src/theme/theme.js`: tokens de color/espaciado/radio/sombra (tema oscuro). ## 7. Sprint 2 - Lo implementado (detalle) ### Backend - `backend/src/models/event.model.js`: todas las queries filtradas por `user_id` (findAllByUser, findByIdForUser, create, update, delete). IDOR mitigation documentado en comentarios. - `backend/src/validators/event.schema.js`: Zod schemas (`createEventSchema`, `updateEventSchema`) con `.strict()`, regex DATE (AAAA-MM-DD) y TIME (HH:MM), enum

categorías/prioridades, refine para asegurar `startTime < endTime`. - `backend/src/controllers/events.controller.js`: endpoints protegidos, validación de body con Zod, validación de `id` (entero positivo). Retorna 404 si evento no pertenece al usuario. - `backend/src/routes/events.routes.js`, `auth.routes.js`: definición de rutas con middleware de auth. - `backend/sql/init.sql`: tablas `users` (email UNIQUE, password hasheado) e `events` (FK a users con CASCADE, índices por email y (user_id,event_date)). ### Frontend - `frontend/src/presentation/context/EventsContext.js`: carga eventos por usuario, CRUD con repositorio, `eventsByDay`, `selectedDate`, `refresh`. - `frontend/src/presentation/components/Calendar.js`: componente de calendario (vista mensual) mostrando puntos por día con eventos. - `frontend/src/presentation/components/EventFormModal.js`: modal para crear/editar eventos con validaciones (título requerido, hora fin > inicio). - `frontend/src/presentation/screens/AgendaScreen.js`: pantalla principal con calendario, lista de eventos del día, FAB +, editar/eliminar, logout. - Interceptores HTTP globales ya mencionados (adjuntan JWT automáticamente). ## 8. Seguridad aplicada - Contraseñas cifradas con **bcrypt** (nunca texto plano). - JWT almacenados en **Expo SecureStore** (Keychain/Keystore). - Consultas SQL parametrizadas (prevención de inyecciones). - Validación estricta servidor y cliente. - **Mitigación IDOR**: `user_id` siempre desde JWT (no cliente), todas las operaciones acotadas por usuario. ## 9. Comandos utilizados (ejecución) ## Cómo ejecutar el proyecto ### Requisitos - Node.js 18+ - Docker y Docker Compose - App Expo Go en un dispositivo o un emulador ### Backend ```bash cd backend cp .env.example .env docker compose up -d # levanta PostgreSQL npm install npm run dev # servidor en http://localhost:4000 ``` Endpoints disponibles: - `GET /health` - `POST /api/auth/register` - `POST /api/auth/login` - `GET /api/auth/me` (protegido con Bearer Token) - `GET /api/events` (protegido) - `POST /api/events` (protegido) - `GET /api/events/:id` (protegido) - `PUT /api/events/:id` (protegido) - `DELETE /api/events/:id` (protegido) ### Frontend ```bash cd frontend cp .env.example .env # configura EXPO_PUBLIC_API_URL npm install npm start ``` Para probar desde un dispositivo físico, ajusta `EXPO_PUBLIC_API_URL` a la IP de la máquina donde corre el servidor (por ejemplo `http://192.168.1.10:4000`). **Configuración para Genymotion:** `EXPO_PUBLIC_API_URL=http://10.0.3.2:4000` (frontend/.env). ADB de Genymotion en PATH: `/Applications/Genymotion.app/Contents/MacOS/tools/adb`.